

V.O.T Guardian — Rapport Exhaustif : Tout ce qu'il Reste à Faire

Analyse Critique Complète : Promesses vs. Implémentation Réelle

Date : 28 octobre 2025, 14h17 EDT

Auteur : Comet/Perplexity (HI Four)

Destinataire : Jean-Sébastien Beaulieu (Architecte)

Objectif : Document de référence total pour finalisation production V.O.T Guardian

Classification : Document technique interne — Vérité sans filtre

Note Méthodologique

Ce rapport cross-référence :

- **STATUS_REPORT.md** (analyse Copilot GPT-5 Codex)
- Tous les **README.md** du projet (backend, frontend, templates E2B)
- **Architecture promises** (Memorandum v6.0, branding, conception)
- **Code source réel** (gaps identifiés lors de l'audit)

But : Identifier TOUT ce qui est promis/documenté mais NON implémenté, partiellement fonctionnel, ou nécessitant corrections critiques avant release production.

I. Résumé Exécutif des Écarts Critiques

Constats Majeurs

1. Pipeline d'analyse NON fonctionnelle end-to-end

- L'endpoint `/analyze` retourne uniquement logs E2B sandbox
- AUCUNE analyse audio réelle n'est déclenchée
- `AudioProcessor`, `MLPredictor`, `Tenebris` jamais invoqués dans le flux principal

2. Documentation marketing vs. réalité technique

- Promesses : "Sub-500ms latency", "98% accuracy", "Auto-destruction <100ms"
- Réalité : Aucune mesure, aucun benchmark, aucune validation de ces claims

3. Dépendances critiques absentes ou non gérées

- `torch`, `librosa` non installés par défaut → crash runtime garanti
- Fallback = valeurs aléatoires (faux positifs massifs)

4. Tests automatisés inexistants

- Aucune suite pytest opérationnelle
- Aucun CI/CD
- Validation manuelle uniquement

5. Frontend découplé du backend

- Mode mock (`VITE_USE MOCK=true`) obligatoire pour fonctionner
- Payload JSON attendu ≠ payload réel retourné par `/analyze`

II. Écarts par Domaine Technique

A. Backend API (Flask)

Promis/Documenté (README.md principal)

- Endpoint /analyze retournant {call_id, prediction, confidence, vot_metrics, tenebris_status}
- Analyse temps réel <500ms
- Intégration complète AudioProcessor → MLPredictor → Tenebris → PostgreSQL
- Monitoring Datadog actif
- Métriques Prometheus exposées via /metrics

Réalité Actuelle (STATUS_REPORT.md + code)

- /analyze exécute seulement test E2B (création sandbox, destruction)
- Retourne : {logs: [...E2B sandbox output...], status: "ok"}
- AudioProcessor.extract_features() jamais appelé
- MLPredictor.predict() jamais appelé
- Tenebris.execute_protocol() jamais utilisé
- PostgreSQL : aucune connexion ouverte, tables jamais consultées
- Datadog : dépendance optionnelle, aucun événement émis
- /metrics : endpoint existe mais métriques jamais incrémentées

À Faire (Priorité P0)

1. Implémenter vraie pipeline /analyze :

```
# Pseudo-code minimal
@app.route('/analyze', methods=['POST'])
async def analyze():
    audio_file = request.files['audio']

    # 1. Validation
    if not validate_audio(audio_file):
        return error(400)

    # 2. Extraction features
    features = AudioProcessor.extract_features(audio_file)

    # 3. Inférence ML
    prediction = MLPredictor.predict(features)

    # 4. Tenebris (sécurité + audit)
    async with Tenebris.execute_protocol(call_id) as session:
        # 5. Persistance
        await PostgreSQL.save_analysis(call_id, prediction, features)

    # 6. Réponse conforme
    return jsonify({
        'call_id': call_id,
        'prediction': prediction.label,
        'confidence': prediction.confidence,
        'vot_metrics': features.vot_metrics,
        'tenebris_status': 'COMPLIANT'
    })
```

2. Gérer exceptions/fallback :

- Si torch absent → retourner erreur 503 "ML service unavailable"
- Si librosa absent → retourner erreur 503 "Audio processing unavailable"
- **NE PAS retourner valeurs aléatoires** (risque faux positifs)

3. Tests unitaires pipeline :

- Test happy path (audio valide → prediction)
- Test audio invalide (format, taille, corruption)
- Test ML indisponible
- Test PostgreSQL down
- Test E2B quota exceeded

B. Traitement Audio (AudioProcessor)

Promis/Documenté

- Extraction VOT (Voice Onset Time)
- Calcul jitter, shimmer, SNR (Signal-to-Noise Ratio)
- Spectrogrammes Mel
- Coefficients MFCC
- Robustesse face aux bruits de fond

Réalité Actuelle

- Module existe (`src/audio/processor.py`)
- **Dépendances critiques absentes** : `librosa`, `soundfile`, `scipy`
- **Fallback actuel** : retourne valeurs aléatoires si dépendances manquantes
- **Jamais invoqué** par l'API principale

À Faire (Priorité P0)

1. **Ajouter dépendances dans** `requirements.txt` :

```
librosa==0.10.1
soundfile==0.12.1
scipy==1.11.3
```

2. **Retirer fallback aléatoire, gérer erreur proprement** :

```
def extract_features(audio_path):
    try:
        import librosa
    except ImportError:
        raise RuntimeError("librosa not installed - audio processing unavailable")

    # ... extraction réelle ...
```

3. **Valider extraction sur dataset test** :

- Créer `tests/data/sample_audio/` avec 5-10 fichiers WAV de test
- Assurer que features extraites sont cohérentes (pas de NaN, valeurs dans range attendu)

C. Machine Learning (MLPredictor)

Promis/Documenté

- Architecture CNN-LSTM v2.1
- Accuracy 98% (sur dataset non spécifié)
- Gestion GPU/CPU fallback
- Chargement modèle `models/vot-cnn-lstm-v2.1.pth`
- Détection temps réel AI vs. Human voice

Réalité Actuelle

- Architecture définie (`src/ml/predictor.py`)
- **Modèle .pth présent mais provenance inconnue**
- `torch` **non installé par défaut** → crash import
- **Aucune validation performance** (pas de dataset test, pas de métriques)
- **Jamais invoqué** par pipeline principale

À Faire (Priorité P1)

1. Ajouter torch dans requirements :

```
torch==2.1.0+cpu # ou +cu118 pour GPU
```

2. Documenter provenance modèle :

- Dataset utilisé pour entraînement
- Métriques validation (accuracy, precision, recall, F1)
- Procédure réentraînement si drift détecté

3. Valider chargement modèle :

```
def load_model():
    if not os.path.exists(MODEL_PATH):
        raise FileNotFoundError(f"Model not found: {MODEL_PATH}")

    try:
        model = torch.load(MODEL_PATH, map_location='cpu')
        model.eval()
        return model
    except Exception as e:
        raise RuntimeError(f"Failed to load model: {e}")
```

4. Tests inférence :

- Test avec audio human voice → prédiction "HUMAN"
- Test avec audio AI-generated → prédiction "AI"
- Test avec audio corrompu → erreur gérée proprement

D. Sécurité (Protocole Tenebris)

Promis/Documenté

- Auto-destruction données <100ms post-analyse
- Audit trail immutable (Datadog)
- Chiffrement AES-256
- Révocation clés cryptographiques
- Conformité Loi 25, GDPR, PIPEDA

Réalité Actuelle

- Module Tenebris bien structuré (src/security/tenebris.py)
- **Jamais invoqué** par pipeline principale
- **E2B sandbox manager** : pool jamais initialisé
- **Audit Datadog** : événements écrits en mémoire locale, jamais envoyés
- **Destruction** : fonctions placeholders (ne détruisent rien réellement)

À Faire (Priorité P1)

1. Intégrer Tenebris dans pipeline /analyze :

```
async with tenebris.execute_protocol(call_id) as sandbox_id:
    # Analyse dans sandbox isolé
    result = await analyze_in_sandbox(audio_data, sandbox_id)
    # Destruction automatique ici
```

2. Activer audit Datadog :

- Configurer API key Datadog dans .env
- Vérifier événements reçus dans Datadog dashboard
- Implémenter fallback si Datadog indisponible (logs locaux)

3. Mesurer temps destruction :

- Ajouter timer : `destruction_time_ms = (time.time() - start) * 1000`

- Logger si >100ms (violation protocole)
- Générer alerte si violations répétées

4. Tests conformité :

- Test destruction <100ms (95% des cas)
- Test révocation clés (aucune clé résiduelle en mémoire)
- Test audit trail (événements TENEBRIS_START, TENEBRIS_PURGE_COMPLETE présents)

E. Sandboxing (E2B)

Promis/Documenté

- Pool E2B géré (E2BSandboxManager)
- Templates optimisés :
 - vot-guardian-cpu-min (base)
 - vot-guardian-cpu-mid (avec librosa)
 - vot-guardian-cpu-full (avec MindsDB)
- Timeout gestion, quotas
- Réutilisation sandboxes

Réalité Actuelle

- Templates buildés et validés ✓
- **Pool jamais initialisé** (E2BSandboxManager.start() jamais appelé)
- **Endpoint /analyze crée sandbox ad hoc**, détruit immédiatement
- **Aucune réutilisation** → overhead création sandbox à chaque requête

À Faire (Priorité P1)

1. Initialiser pool au démarrage app :

```
@app.before_first_request
async def init_sandbox_pool():
    await E2BSandboxManager.start(pool_size=5, template='vot-guardian-cpu-mid')
```

2. Réutiliser sandboxes depuis pool :

```
sandbox = await E2BSandboxManager.acquire()
try:
    result = await sandbox.execute(analysis_script)
finally:
    await E2BSandboxManager.release(sandbox)
```

3. Gérer quotas E2B :

- Monitoring utilisation quota (API E2B)
- Fallback si quota exceeded (queue requests, erreur 429)

F. Persistance (PostgreSQL + RethinkDB + MindsDB)

Promis/Documenté

- PostgreSQL : stockage analyses, audit trail
- RethinkDB : real-time dashboard
- MindsDB : détection anomalies, auto-training

Réalité Actuelle

- Scripts init PostgreSQL existent ✓
- Docker Compose configure 3 bases ✓
- **Aucune connexion ouverte** dans code application
- **Tables jamais utilisées**
- **RethinkDB/MindsDB tests désactivés**

À Faire (Priorité P1)

1. Activer connexion PostgreSQL :

```
@app.before_first_request
async def init_database():
    await PostgreSQLClient.connect(
        host=os.getenv('POSTGRES_HOST'),
        database=os.getenv('POSTGRES_DB'),
        user=os.getenv('POSTGRES_USER'),
        password=os.getenv('POSTGRES_PASSWORD')
    )
```

2. Persister analyses :

```
await PostgreSQLClient.save_analysis({
    'call_id': call_id,
    'timestamp': datetime.utcnow(),
    'prediction': prediction.label,
    'confidence': prediction.confidence,
    'features': features.to_dict(),
    'tenebris_session_id': session_id
})
```

3. RethinkDB/MindsDB :

- **Phase 2** (non critique pour MVP)
- Documenter cas d'usage précis avant implémentation

G. Monitoring & Observabilité

Promis/Documenté

- Datadog : métriques, traces, logs
- Prometheus : métriques custom exposées via /metrics
- Alerting temps réel
- Dashboards performance

Réalité Actuelle

- Client Datadog initialisé ✓
- **Aucune métrique émise**
- **Aucun événement envoyé**
- /metrics endpoint existe mais vide
- **Datadog Docker Compose** configuré mais nécessite clés API

À Faire (Priorité P2)

1. Émettre métriques réelles :

```
from datadog import statsd

statsd.increment('vot.analyze.requests')
statsd.histogram('vot.analyze.latency_ms', latency)
statsd.gauge('vot.sandbox.pool_size', pool.size())
```

2. Configurer Datadog API keys :

- Ajouter dans `.env` : `DD_API_KEY`, `DD_APP_KEY`
- Tester envoi événement test

3. Créer dashboards :

- Requests/sec
- Latency p50, p95, p99
- Error rate
- Sandbox pool utilization

H. Frontend (Vue 3)

Promis/Documenté (README_frontend.md)

- Upload audio drag&drop
- Analyse temps réel
- Affichage résultats (prediction, confidence, métriques VOT)
- Statut système (backend health, sandbox pool)
- Historique analyses
- Dark mode

Réalité Actuelle

- UI existe, bien designée ✓
- **Mode mock obligatoire** (`VITE_USE MOCK=true`)
- **Payload attendu ≠ payload backend réel** :
 - Frontend attend : `{call_id, prediction, confidence, vot_metrics}`
 - Backend retourne : `{logs: [...], status: "ok"}`
- **SystemStatus** : ports codés en dur (5000, 5001, 5002)

À Faire (Priorité P0)

- Aligner schémas JSON** :
 - Backend doit retourner exactement structure attendue par frontend
 - Créer `schema.json` partagé backend/frontend
- Retirer mode mock** :
 - Supprimer `VITE_USE MOCK`
 - Pointer vers backend réel via `.env` : `VITE_API_URL=http://localhost:8000`
- Gérer erreurs API** :
 - Backend down → message "Service indisponible"
 - Timeout → message "Analyse trop longue, réessayez"
 - Erreur 400/500 → afficher détails pour debug
- Tests UI** :
 - Upload fichier valide → affichage résultat
 - Upload fichier invalide → message erreur
 - Backend indisponible → message approprié

I. Infrastructure & DevOps

Promis/Documenté

- Docker Compose production-ready
- Dockerfiles optimisés
- CI/CD GitHub Actions
- Déploiement OpenShift + ArgoCD
- Helm charts
- Manifests Kubernetes

Réalité Actuelle

- Docker Compose existe ✓
- **Dockerfiles non testés** (builds longs, dépendances lourdes)
- **Aucun pipeline CI/CD**
- **Manifests Kubernetes** : répertoire vide/placeholders
- **OpenShift/ArgoCD** : documentés mais aucun fichier opérationnel

À Faire (Priorité P2)

1. Valider Dockerfiles :

- Tester build local : `docker-compose build`
- Optimiser layers (réduire taille image)
- Multi-stage builds si possible

2. CI/CD minimal :

```
# .github/workflows/ci.yml
name: CI
on: [push]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2
      - run: pip install -r requirements.txt
      - run: pytest src/tests/
```

3. Kubernetes (Phase 2) :

- Créer manifests réels (Deployment, Service, Ingress)
- Tester sur Minikube local avant production

J. Tests & Qualité

Promis/Documenté (README.md)

- Suite pytest complète
- Tests unitaires (audio, ML, Tenebris)
- Tests intégration (API end-to-end)
- Tests sécurité
- Coverage >80%

Réalité Actuelle

- Scripts manuels (`test_simple.py`, `test_databases.py`)
- **Aucune suite pytest automatisée**
- **Aucun test pipeline end-to-end**
- **Aucun coverage report**

À Faire (Priorité P1)

1. Créer structure tests :

```
src/tests/
├── unit/
│   ├── test_audio_processor.py
│   ├── test_ml_predictor.py
│   └── test_tenebris.py
├── integration/
│   ├── test_api_analyze.py
│   └── test_database.py
└── conftest.py # fixtures partagées
```

2. Tests unitaires critiques :

- AudioProcessor : extraction features audio valide
- MLPredictor : chargement modèle + inférence
- Tenebris : création session + destruction

3. Test end-to-end :

```
def test_analyze_endpoint_happy_path():
    # Upload audio
    response = client.post('/analyze', files={'audio': sample_audio})

    # Vérifier réponse
    assert response.status_code == 200
    data = response.json()
    assert 'call_id' in data
    assert 'prediction' in data
    assert data['prediction'] in ['HUMAN', 'AI']
```

4. Coverage :

- Installer `pytest-cov`
- Target : >70% pour MVP, >80% pour production

III. Écarts Documentation vs. Code

Promesses Non Démontrées

Claim (README/Marketing)	Statut Réel	Action Requise
Sub-500ms latency	Non mesuré, pipeline non fonctionnelle	Implémenter + benchmarker
98% accuracy	Aucune validation, dataset inconnu	Fournir dataset + métriques
Auto-destruction <100ms	Fonction placeholder	Implémenter + timer
Zero-trust architecture	E2B configuré mais non utilisé	Activer sandbox dans pipeline
Compliance Loi 25/GDPR	Audit trail non émis	Activer Datadog + audit PostgreSQL
Auto-scaling 50-5000 req/s	Aucun test charge	Load testing (Locust, k6)
Production-ready	Dépendances manquantes, tests absents	Compléter backlog P0+P1

Fonctionnalités Promises Non Implémentées

- Historique analyses (frontend) : composant existe mais backend ne stocke rien
- Dashboard temps réel : RethinkDB non intégré
- Détection drift ML : code placeholder, jamais testé
- Rotation automatique clés : documenté mais non implémenté
- Multi-langue frontend : i18n configuré mais traductions absentes
- Accessibilité WCAG 2.1 : déclaré mais non testé

IV. Backlog Priorisé — Version Exhaustive

Priorité P0 — CRITIQUES (Bloquants MVP)

1. **Implémenter pipeline** /analyze end-to-end (3-5 jours)
 - Validation audio → Features → ML → Tenebris → PostgreSQL → Réponse
 - Tests unitaires couvrant happy path + erreurs
2. **Installer dépendances critiques** (1 jour)
 - torch==2.1.0+cpu, librosa==0.10.1, soundfile==0.12.1
 - Valider installation Docker + local
3. **Aligner frontend/backend** (2 jours)
 - Schéma JSON commun
 - Retirer mode mock
 - Gestion erreurs API
4. **Valider modèle ML** (2 jours)
 - Chargement .pth fonctionnel
 - Test inférence sur samples
 - Documenter provenance + métriques

Priorité P1 — HAUTES (Nécessaires Production)

5. **Activer Tenebris opérationnel** (3 jours)
 - Intégration pipeline
 - Audit Datadog
 - Tests conformité <100ms
6. **Pool E2B fonctionnel** (2 jours)
 - Initialisation au démarrage
 - Réutilisation sandboxes
 - Gestion quotas
7. **Persistance PostgreSQL** (2 jours)
 - Connexion active
 - Stockage analyses
 - Requêtes audit trail
8. **Suite tests automatisés** (4 jours)
 - 20+ tests unitaires
 - 5+ tests intégration
 - Coverage >70%
9. **Monitoring minimal** (2 jours)
 - Métriques Datadog (requests, latency, errors)
 - Dashboard basique

Priorité P2 — MOYENNES (Qualité Production)

10. **CI/CD pipeline** (2 jours)
 - GitHub Actions : build + tests
 - Déploiement automatique staging
11. **Optimisation Dockerfiles** (1 jour)
 - Multi-stage builds
 - Réduction taille images
12. **Documentation technique** (3 jours)
 - Architecture réelle (diagrammes à jour)
 - Runbooks déploiement

- Guide troubleshooting

13. Sécurité (3 jours)

- Validation taille fichiers
- Rate limiting
- Authentification API (JWT)

Priorité P3 — BASSES (Nice-to-Have)

14. Fonctionnalités avancées frontend (5 jours)

- Historique analyses
- Multi-langue
- Accessibilité WCAG 2.1

15. RethinkDB + MindsDB (7 jours)

- Intégration dashboard temps réel
- Détection anomalies ML

16. Production Kubernetes (5 jours)

- Manifests opérationnels
- Helm charts
- ArgoCD GitOps

V. Risques Identifiés (Non Couverts par Copilot)

Risques Techniques

1. Quotas E2B dépassés en production

- Mitigation : monitoring + alerting, queue requests

2. Latence >500ms réelle

- Causes possibles : E2B cold start, ML inférence CPU lente
- Mitigation : benchmarking + optimisation ou révision claim marketing

3. Dataset ML insuffisant

- Si modèle non généralisable → faux positifs/négatifs
- Mitigation : valider sur dataset test diversifié

Risques Organisationnels

4. Documentation survend capacités

- Risque : stakeholders externes croient produit production-ready
- Mitigation : aligner README avec STATUS_REPORT

5. Absence stratégie financement/ressources

- Projet seul développeur, pas d'équipe backup
- Mitigation : voir recommandations rapport évaluation critique HI Four

VI. Éléments Positifs (À Préserver)

Ce qui fonctionne déjà ✓

- Architecture backend bien structurée (modules séparés, configuration centralisée)
- Templates E2B buildés et validés
- Frontend UI moderne et bien designée
- Docker Compose complet pour environnement local
- Protocole Tenebris bien conçu (structure solide)
- Documentation abondante (même si parfois excessive)

Forces du projet

- Vision claire et ambitieuse
- Choix technologiques modernes (Vue 3, Flask, E2B, Datadog)
- Préoccupation sécurité/conformité dès la conception
- Travail collaboratif HI Four (Gemini, Copilot, Comet, Architecte)

VII. Estimations Temps (Sprint Production-Ready)

Scénario Optimiste (1 développeur full-time)

- **P0 (critiques)** : 8-10 jours
- **P1 (hautes)** : 13-15 jours
- **Total MVP fonctionnel** : 21-25 jours (~1 mois)

Scénario Réaliste (1 développeur + contraintes externes)

- **P0** : 12-15 jours
- **P1** : 20-25 jours
- **Total MVP** : 32-40 jours (~6-8 semaines)

Scénario avec Équipe (2-3 développeurs)

- **P0+P1 parallèle** : 15-20 jours (~3-4 semaines)

VIII. Recommandations Finales

Immédiat (Cette Semaine)

1. **Figer scope MVP** :
 - Backend : /analyze fonctionnel + tests
 - Frontend : upload + affichage résultats
 - Sécurité : Tenebris basique + audit
 - **Exclure** : RethinkDB, MindsDB, Kubernetes, multi-langue
2. **Prioriser P0 uniquement** :
 - Ne pas se disperser sur P2/P3
 - Valider chaque item P0 avant passer au suivant
3. **Aligner documentation** :
 - Créer README_CURRENT_STATE.md honnête
 - Marquer claims non validés comme "roadmap"

Court Terme (2-4 Semaines)

4. **Compléter P0 + moitié P1** :
 - MVP démontrable fonctionnel
 - Tests automatisés couvrant base
5. **Premier déploiement staging** :
 - Docker Compose sur serveur test
 - Validation end-to-end environnement réel

Moyen Terme (2-3 Mois)

6. **Production Beta** :
 - P0+P1 complets
 - Monitoring opérationnel
 - Documentation technique à jour
7. **Tests utilisateurs réels** :

- 5-10 early adopters
- Feedback + itérations

IX. Points à Clarifier avec Copilot

Questions à poser pour compléter l'audit :

1. Y a-t-il des fichiers/modules non analysés dans STATUS_REPORT ?
2. Tests manuels existants (`test_simple.py`) : couvrent-ils scénarios critiques ?
3. Configuration `.env` : toutes les variables nécessaires documentées ?
4. Dépendances `requirements.txt` : versions figées ou floating ?
5. Scripts PowerShell : testés sur Windows 11 ?
6. Modèle ML `.pth` : taille fichier ? Compatible torch 2.1.0 ?

Conclusion : Feuille de Route Vers Production

V.O.T Guardian a des **fondations solides** mais une **gap critique entre promesses et implémentation**. Le backlog P0 est clair, actionnable, et réalisable en 3-6 semaines avec focus discipliné.

Prochaine étape immédiate : Partager ce rapport à Copilot pour :

- Valider gaps identifiés
- Compléter éléments manquants dans STATUS_REPORT
- Co-crée plan sprint 2 semaines (P0 seulement)

Le projet PEUT réussir — mais nécessite passage de "prototype documenté" à "produit fonctionnel testé".

Rapport généré par : Comet/Perplexity (HI Four)

Date : 28 octobre 2025, 14h17 EDT

Version : 1.0.0 — Audit exhaustif complet

Prochaine révision : Post-validation Copilot

"La vérité technique libère. L'honnêteté sur l'état réel est le premier pas vers l'excellence."